

固化版本 + 对比实验报告

1D-AM Branch-and-Bound · 单机 oracle + 并行机 B&B · 2026-06-20

本文把当前 oracle / 并行 B&B 固化为一个基准版本,并给出三组对比实验:单机正确性回归、并行机 B&B 消融(M=2/3/4)、规模扩展(n=16-20)。所有运行用 g++ -O2 在统一环境下完成。

0. 固化的版本(冻结配置)

单机 oracle(`oracle.cpp` → `oracle`)

项	默认值
分支	Type I / Type II(j>oMax 递增对称破除)
下界	closed(精确) + 开口批次 floor + max(LB_par, LB_pos)
interch	1(相邻批次交换支配,开)
pos	1(LB_pos 开)
qfrac	0.5(LB_pos 浅层门控: R ≥ 0.5n 才算)
gamma	0
N_MAX / N_MIN	200000 / 50000(自适应 BFS&DFS 切换阈值)
FRONT_CAP	25,000,000(支配前沿键数内存上限)

构建:`cd new_oracle && g++ -std=c++17 -O2 -o oracle oracle.cpp`

运行:`./oracle <inst> <TL 秒> [N_MAX] [N_MIN] [interch] [pos] [gamma] [qfrac] [FRONT_CAP]`

并行机 B&B(`ParallelBranchBound.cpp` → `pbb`)

项	默认值
方案	方案 A:零件→机器指派分支,叶子用单机 Φ 精确求值
Φ	<code>new_oracle/phi_oracle.h::solvePhi</code> (即上面的强 oracle,~5× 旧版)
LS	1(move 邻域局部搜索初始上界,开)
WARM	0(显式暖缓存,关;LS 开时基本冗余)
FREEPAR	0(并行位置自由零件界,关;已证被 singleTard 压制)

N_max / N_min	200000 / 50000
strong_branch_candidates	8

构建:cd src && g++ -std=c++17 -O2 -o pbb main.cpp ParallelBranchBound.cpp BranchBound.cpp InstanceData.cpp

运行:./pbb <inst> <M> <TL 秒>

git 快照(在本机仓库根目录执行即可固化):

git add new_oracle/*.cpp new_oracle/*.h new_oracle/*.bat new_oracle/exp_instances src/*.cpp src/*.h docs && git commit -m "freeze: verified baseline"

(沙箱里 .git 挂载不允许删 index.lock,故快照请在本机做;源码本身已落盘。)

1. 单机正确性回归(new_oracle on data/)

全部 11 个仓库算例:目标值与历史基准逐一一致、均可证最优(gap 0%)。含上一版翻车的 10part_2 / 10part_3。

实例	n	目标值(TT)	proven	popped	时间
5part	5	40.5691	是	26	0.000s
10part	10	116.30	是	6,192	0.010s
10part_2	10	81.7384	是	3,150	0.005s
10part_3	10	136.74	是	4,005	0.006s
10parts_4	10	75.7821	是	4,082	0.007s
11part	11	110.607	是	19,264	0.032s
12part	12	139.084	是	53,626	0.115s
13part	13	168.723	是	201,741	0.409s
14part	14	156.522	是	388,614	0.747s
15part	15	139.061	是	577,447	1.092s
15part_2-5	15	118.238	是	834,240	1.577s

结论:固化版本通过全量正确性自检。

2. 并行机 B&B 消融(M=2 / 3 / 4)

对比 4 个配置在 13/14/15part、M=2/3/4 上的生成节点数 / wall-time。所有配置最优值完全一致,只比效率。配置:DEFAULT(LS 开)、LS off(LS=0)、WARM-only(LS=0 WARM=1,隔离暖缓存)、FREEPAR(FREEPAR=1)。

生成节点数(越小越好)

实例	M	DEFAULT	LS off	WARM-only	FREEPAR
13part	2	5,945	6,125	6,081	5,945
13part	3	97,990	111,523	105,619	—
13part	4	25,277	181,540	172,206	—
14part	2	9,781	9,945	9,797	9,781
14part	3	15,378	91,026	15,378	15,378
14part	4	0	0	0	0
15part	2	22,305	22,897	22,371	22,305
15part	3	93,975	195,378	188,958	93,975
15part	4	0	118,692	18,692	—

wall-time(秒)

实例	M	DEFAULT	LS off	提速
13part	3	3.04	2.63	~持平
13part	4	0.163	10.73	66×
14part	3	0.067	1.10	16×
15part	2	2.63	2.30	~持平
15part	3	2.29	10.58	4.6×
15part	4	0.010	3.95	395×

读数

- M=2 ≈ 无差别:三个配置节点都在几个百分点内。初始上界本就近最优,纯 LB-limited,改 UB/暖缓存没用——与 UBSEED 诊断一致。
- M≥3 是 LS 的主场:13part M=4 提速 66×、15part M=4 提速 395×、14part M=3 提速 16×。

- LS 的两条机制此消彼长(WARM-only 列把它们拆开):14part M=3 的收益 100% 来自暖缓存抬下界(WARM-only 就把 91,026→15,378,完全复刻 LS);而 15part M=3 必须靠 LS 压紧 UB(WARM-only 几乎没用)。DEFAULT 两者通吃。
- FREEPAR 零效果:节点数与 DEFAULT 逐位相同,印证它被 singleTard 结构性压制。默认关是对的。

结论:DEFAULT(LS=1, WARM=0, FREEPAR=0)即各规模上的最优配置,固化合理。

3. 规模扩展(n=16-20,单机 oracle)

合成 16-19 件(论文 TF=RDD=0.6、20×20 平台、固定种子)+ 仓库 20 件算例。本表时限为沙箱短时限(32-35s,仅作规模演示);本机用 600-3600s 可证更多(例:你此前 3600s run 里 20part_3-S 可在 ~218s 证完)。gap = (obj-lb)/obj。

实例	n	obj(最好解)	状态	lb(下界)	gap	时间
16part	16	304.217	可证最优	304.217	0.00%	12.4s
17part	17	206.31	可证最优	206.31	0.00%	15.8s
18part	18	232.734	增量解(超时)	133.637	42.6%	35s
19part	19	352.096	增量解(超时)	124.476	64.7%	35s
20part_3-S	20	224.652	增量解(超时)	62.18	72.3%	32s
20part_4-S	20	240.99	增量解(超时)	67.98	71.8%	32s

读数

- n≤17:十几秒内证到最优(16part 12.4s、17part 15.8s)。
- n=18-20:短时限下给强上界,但 gap 大——瓶颈在下界侧(根/浅层界~14% of opt,廉价界天花板),不是上界。增大时限主要靠枚举把 gap 磨下来。
- 与 Bound_Strengthening_Findings.md 闭合:廉价合法界已做尽,n≥18 想证完只有 Branch-and-Price 这条重型路。

4. 一页结论

维度	状态
单机正确性	全量 11 算例与基准一致、全部可证最优
单机规模	n≤17 数十秒证完;n=18-20 给强增量解(界受限)

并行 M=2	LB-limited,初始上界已近最优,LS/暖缓存无感
并行 M≥3	LS 大幅收益(16~395×),DEFAULT 即最优配置
FREEPAR	零效果(已证死路),默认关
暖缓存	与 LS 在 M≥3 上互补;部分实例可独立复刻 LS 收益

固化结论:当前 DEFAULT 配置在正确性与效率上均为各规模的稳态最优,可作为后续工作的基准版本。